answer.py 逐行翻译：一道题从检索到回答，中间发生了什么

七十三行代码，只有三个动作：找料、判断、作答。
难的不是写出来，是知道每一步在哪一层负责。

睡前 3 分钟版

191	3	6.3s
全部行数 5 个函数	道闸门 检索 / 阈值 / prompt	单题耗时 其中模型约 2s

- **它不是「调用模型」，是「给模型准备好材料，再逼它只说材料里的话」。**模型在这套系统里是最不重要的一环。
- **阈值判断必须放在调 API 之前。**放后面就是花完钱再拒，白烧 token。
- **返回 `dict` 而不是字符串。**因为一次回答里有五样东西要交付：结论、证据、分数、引用、是否拒答。
- **没装 openai SDK。**一个 POST 就能干的事，没必要多一个依赖。

01 数据流：五个动作

① 问题 → **search()** → 3 块材料 + 3 个分数（复用第 ④ 站的精排版，不重写）

② 看第 1 块分数 → **低于 0.60 直接拒答**，连 API 都不调

③ 3 块材料 → **编号 [1][2][3] + 去掉 ## 前缀** → 拼成【材料】段落

④ 系统提示 + 材料 + 问题 → **一个 POST 到 deepseek**

⑤ 答案里的 **[n]** 抽出来反查 → 有没有越界乱标

五步里有四步跟模型无关。**RAG 的功夫全在模型之外**，模型只负责把材料组织成人话。

02 完整代码（带逻辑标注）

第一段：配置与宪法

```
TOP_K = 3               # 喂给模型几段材料
MIN_SCORE = 0.60        # 精排分低于这个就拒答（abstain.py 扫出来的最优阈值）
API_URL = "https://api.deepseek.com/chat/completions"
MODEL = "deepseek-chat"

# 这段话是整个系统的「宪法」。没有它，模型见到材料就会开始自由发挥。
SYSTEM_PROMPT = (
    "你是一个只根据给定材料回答问题的助手，必须遵守以下规则：\n"
    "1. 只能使用【材料】里的内容回答，不许用自己的知识补充，也不许推测。\n"
    "2. 每句结论后面用 [1] [2] [3] 标出它出自哪段材料；用到几段就标几个。\n"
    "3. 如果材料里没有足够信息回答问题，直接回答「材料里没有相关内容」，不要硬凑。\n"
    "4. 回答控制在 3 句话以内，直接给结论，不要复述问题。"
)
```

第 3 条是全文最重要的一句。它把「拒答」这件事从检索层挪到了生成层——因为检索分数测的是*语义相关*，不是*能不能回答*。书里只提了个名字的题，检索分能到 0.969，只有模型读了材料才知道够不够。

第二段：把检索结果拼成材料

```
def build_material(hits):
    """每段材料带编号，编号就是后面引用的锚点"""
    parts = []
    for n, (idx, score) in enumerate(hits, 1):
        title, body = chunks[idx][1], chunks[idx][2]
        title = title.strip().lstrip("#").strip()
        #        ↑ "## 1.1 什么是智能体"  →  "1.1 什么是智能体"
        parts.append("[%d] 出自《%s》\n%s" % (n, title, body.strip()))
    return "\n\n".join(parts)


def build_prompt(question, hits):
    return "【材料】\n%s\n\n【问题】\n%s" % (build_material(hits), question)
```

> **为什么要把 `##` 去掉**
>
> `##` 对模型是噪音，但「1.1 什么是智能体？」这个标题是有价值的——它让模型知道这段出自哪一节，引用时能说出处。
>
> **切块时产生的元数据，到生成时变成了引用标注。**这条链是闭的：切块记录标题 → 向量化带上标题 → 检索命中标题 → 材料里带上标题 → 答案里标出来源。

第三段：调用模型

```
def ask_llm(prompt, key, temperature=0.1):
    """OpenAI 兼容接口，一个 POST 就够，没必要为它装个 SDK"""
    body = json.dumps({
        "model": MODEL,
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},   # 宪法
            {"role": "user",   "content": prompt},          # 材料 + 问题
        ],
        "temperature": temperature,   # 0.1：要的是忠实复述材料，不是创作
        "max_tokens": 512,
    }).encode("utf-8")

    req = urllib.request.Request(
        API_URL, data=body,
        headers={"Content-Type": "application/json",
                 "Authorization": "Bearer " + key},
    )
    with urllib.request.urlopen(req, timeout=60) as r:
        data = json.load(r)
    return data["choices"][0]["message"]["content"], data.get("usage", {})
```

三层嵌套就是接口的全部形状：`choices[0].message.content`——第一个候选、里面的消息、消息的正文。`usage` 顺手拿回来，是为了看 token 花在哪：**prompt token 太少，说明检索喂的材料本身就很薄**，这比看分数更能反映问题。

第四段：主流程（三个逻辑判断，看懂这段就够）

```
def answer(question, key=None, dry=False):
    """返回一个 dict：hits 是证据，answer 是结论，refused 表示兜没兜住"""
    hits = search(question, k=TOP_K)        ◀ 逻辑 1：检索只管找，不管判
    top = hits[0][1]                        ◀ 逻辑 2：取第一名分数（不是第一个结果）

    if top < MIN_SCORE:                     ◀ 逻辑 3：调 API 之前先拦一道
        return {
            "question": question, "hits": hits, "answer": None,
            "refused": True,
            "reason": "top1 精排 %.3f < 阈值 %.2f" % (top, MIN_SCORE),
        }

    prompt = build_prompt(question, hits)
    if dry:                                 ◀ 逻辑 4：dry 模式在调模型前返回，省 key 省时间
        return {"question": question, "hits": hits,
                "answer": None, "refused": False, "prompt": prompt}

    try:
        text, usage = ask_llm(prompt, key or load_key())    ◀ 逻辑 5：短路求值，传了就用传
的
    except urllib.error.HTTPError as e:                     ◀ 逻辑 6：错误也走同一个 return 形状
        detail = e.read().decode("utf-8", "ignore")[:200]
        return {"question": question, "hits": hits, "answer": None,
                "refused": False, "prompt": prompt,
                "error": "接口返回 %d：%s" % (e.code, detail)}

    return {"question": question, "hits": hits, "answer": text.strip(),
            "refused": False, "prompt": prompt, "usage": usage}
```

03 六个逻辑，逐条翻译

逻辑	代码	翻译成人话
1	`hits = search(q, k=3)`	拿回 3 个 `(块下标, 分数)`。**检索不判断能不能答**，它只负责把最像的三块捞上来，判断是后面两道闸门的事。
2	`top = hits[0][1]`	`hits[0]` 是第一名那个元组，`[1]` 是它里面的分数（`[0]` 是块下标）。**取的是分数，不是块。**
3	`if top < 0.60`	第一道闸门。**位置很关键**：放在 `ask_llm` 之前，拒答就不花一分钱；放后面就是烧完 token 再说「不答」。
4	`if dry: return`	调试开关。把 `prompt` 原样吐出来，你可以肉眼检查材料拼得对不对。**调 RAG 时最先该看的不是答案，是 prompt。**
5	`key or load_key()`	短路求值：`key` 非空就用它，空的才去读 `.env`。所以命令行传的 key 优先级高于配置文件。
6	`except HTTPError`	**错误也返回同样的 dict 形状**，多一个 `error` 字段。这样上层调用不用写两种处理分支，`if r.get("error")` 一句就够。

04 返回 dict 而不是字符串

这是整个文件里最容易被写错的设计。如果 `answer()` 直接 `return text`，那么上层就拿不到证据、分数、是否拒答，**出了错也无从定位。**

```
# 一次回答其实要交付五样东西
{
  "answer":  "MCP 统一了智能体与外部工具的交互方式[1]",   # 结论
  "hits":    [(12, 0.985), (33, 0.880), (58, 0.665)],  # 证据（谁支撑的）
  "refused": False,                                     # 是不是拒答
  "prompt":  "【材料】...",                              # 模型实际看到的东西
  "usage":   {"prompt_tokens": 1435, ...},             # 花了多少 token
}
```

有了这五样，出问题的时候你能一层层往回查：答案不对 → 看 `hits` 是不是对的块 → 看 `prompt` 是不是完整 → 看 `usage` 是不是太薄。**只返回一个字符串的话，这四层全断了。**

05 四个容易写错的地方

写法	会怎样	为什么
`chunks[idx][0]`	喂给模型一堆文件路径	元组是 `(path, title, text)`，`[0]` 是路径，`[1]` 才是标题
`title.lstrip("#")`	前面留一个空格	`"## 1.1"` 去掉井号后是 `" 1.1"`，**后面还得再 strip 一次**
`hits[0]`	把元组当分数比大小	要 `hits[0][1]`。`hits[0]` 是 `(下标, 分数)`，拿它跟 0.60 比会报错
`elif r["answer"] is None`	出错时显示成「dry 模式」	`error` 那条分支**必须放在** `answer is None` **之前**，因为出错时 answer 也是 None

> **最后一条是真踩过的**
>
> 写 `show()` 的时候先写了 `elif r["answer"] is None`，再加 `error` 分支就顺手加在了后面。结果 key 错的时候，屏幕上显示「dry 模式，未调用模型」，而实际上模型调了、还失败了。
>
> **分支顺序错了不报错，只会显示错误的信息。**这跟切块那天的 `~~~` 是同一类问题：静默失败。

06 这套代码有一个真实漏洞

今晚做判断练习时发现的：如果材料里只有一段代码，代码里出现了 `stdio` 和 `sse` 两个词，但没有一句话说「这两个是传输方式」，模型会直接答「MCP 支持 stdio 和 sse 两种传输方式 [3]」。

答案碰巧是对的，但推理链不成立。假如代码里写的是 `websocket`，它照样会这么说。

现有宪法管不住这种情况：第 3 条只管「材料不足就说不知道」，管不住「材料提了个词，模型自己升级成结论」。所以还缺一条：

```
"5. 只有材料明确陈述的内容才能作为结论；不许从代码、示例、片段里推断。"
```

> **这条为什么难想到**
>
> 因为它**只在答案正确时才暴露**。答案错的时候你会去查；答案对的时候没人查证据。这类 bug 在线上会一直活下去。
>
> 所以 RAG 的验收标准不能是「答案对不对」，得是**「证据能不能支撑这个结论」**。

07 明天从哪接着来

这套代码本身没有大问题了，卡住的是它上游：

题	用户怎么问	书里叫什么	检索分
8.3	先检索外部资料再让模型生成答案	检索增强生成（RAG）	0.790
4.2	一边推理一边调用工具的范式	ReAct（Reason + Act）	0.854

问题里一个术语都没有，文档全靠术语，**bi-encoder 建不起这座桥**。明天做查询改写：让模型先把口语问题改写成术语问题，再拿去检索。

改写的代码会长在 `answer.py` 的最前面，多一步、多一次调用、多一秒，但能救回的是整类题。

文件：answer.py · 191 行 · 5 个函数 · 3 道闸门 · 1 个待补的漏洞

下一步：查询改写 → HyDE → 再回头处理 PER_SECTION 的配额